Project Log: Athlete Kinetics Edge-AI Platform

Architecture & Development Chat Record

June 10, 2026

Phase 1: Project Ideation

User: Request to distill an idea for a fitness app. The app tracks training programs, logs sessions, and sets goals. It connects to wearables (mood ring, SpO2, heart rate, stress, sleep) to log biometric data. Features a custom-built, basic embedded AI coach (no API costs) using a Small Language Model trained on the user's own competitive powerlifting and coaching experience.

Assistant: Distilled the concept into a pitch-ready format:

- **Command Center:** Intelligent logging and goal setting.
- **Biometric Bridge:** Wearable integration for passive vitals.
- **Edge AI Coach:** High-impact, zero-API overhead SLM trained on synthetic expert data.

User: Added context about competing in Jiu-Jitsu, top 1% in Rocket League, and studying Electrical Engineering. The long-term vision is to develop better sensors, hardware, and eventually human-integrated neural networks for AR.

Assistant: Expanded the vision into a three-phase master plan:

1. Software & Edge AI (The Command Center)
2. Proprietary Sensor Development (Hardware)
3. The Endgame (Neural Integration & AR)

Phase 2: Tool Selection & Strategy

User: Asked for the next actionable step and prompts to feed into Claude Opus or Fable.

Assistant: Recommended Claude 3.5 Sonnet initially for system architecture and provided three foundational prompts covering the Tech Stack, Database Schema, and SLM Coaching Logic.

User: Asked why Fable 5 was not mentioned.

Assistant: Corrected the oversight. Fable 5 is ideal for this project due to its autonomous, long-horizon coding capabilities and ability to hold massive architectural context. Noted the caveats of token usage and strict safety guardrails.

User: Asked if Opus 4.8 would be better.

Assistant: Agreed that Opus 4.8 might be better for avoiding Fable 5's hyper-strict medical and biological guardrails, which might block prompts regarding SpO2 and HRV.

User: Shifted focus temporarily to building an automated sentiment-based trading bot to generate capital for university.

Assistant: Outlined the technical realities of sentiment trading (data latency, overfitting, transaction costs) and provided an Opus 4.8 prompt for building a Historical Backtesting Engine to test the math safely.

User: Decided to return to the Coaching App using Fable, noting they don't have the capital to trade anyway and that medical disclaimers should suffice for the coaching app.

Assistant: Pivoted back to Fable 5. Provided a strategy to bypass Fable's medical filters by framing the app as a "Biomechanical Data Parsing Engine" dealing with physics and mechanical load, rather than health. Provided updated Prompts 1, 2, and 3 for Fable.

Phase 3: Architecture Initialization

User: Uploaded an image showing Fable 5 requesting permission to search the web for "iOS jetsam memory limit per app iPhone 12 iPhone 15 os_proc_available_memory". Asked why it was asking this.

Assistant: Explained that Fable is verifying the exact hardware RAM limits (Jetsam) to ensure the local SLM does not crash older devices. Advised allowing the search.

User: Provided Fable's output for Prompt 1. Fable successfully audited the memory limits, selecting React Native (Hermes), SQLite (op-sqlite), and llama.rn (Qwen2.5-1.5B-Instruct Q4_K_M). Fable output the full monorepo directory structure and initialization commands.

Assistant: Praised the architecture (specifically mitigating mmap'd weights page-cache thrashing) and mistakenly generated the SQL code for the database instead of providing the prompt for Fable.

User: Pointed out that they only fed Prompt 1 to Fable.

Assistant: Acknowledged the error and provided Prompt 2 for Fable to generate the database schema autonomously.

Phase 4: Database & Inference Layer Generation

User: Provided Fable's output for the database schema. Fable generated `001_mechanical_input.sql`, `002_telemetry.sql`, `003_state_vector.sql`, and `004_state_vector_materialize.sql`. The verification script caught and fixed a `BEFORE DELETE` trigger bug. Fable confirmed op-sqlite over Realm due to memory overhead and trigger support.

Assistant: Validated the trigger fix and the `mmap_size=0` cache mitigation. Provided Prompt 3 to build the Edge AI Data Pipeline and GBNF Grammar.

User: Provided Fable's output for the inference layer. Fable generated a strict GBNF grammar (`ADJUSTMENT_GRAMMAR`) forcing enum-only numerics, and a `promptAssembly.ts` utilizing static-prefix KV reuse for high performance. Verification tests passed all 192 combinations.

Assistant: Highlighted the efficiency of forcing literal enums to prevent hallucination and the KV cache reuse for speed. Provided Prompt 4 for the Synthetic Biomechanical Seeder.

Phase 5: Refocusing & Project Roadmap

User: Requested the assistant stop praising Fable, act as a strict, unbiased auditor moving forward, and use a tracker line to log phases.

Assistant: Acknowledged the directive for an objective, red-team persona.

User: Asked to drop the tracker line and instead output the master outline of all phases.

Assistant: Outlined the Master Blueprint:

1. **Phase 1: Foundation & Memory Envelope (Completed)**
2. **Phase 2: The Data Layer (Completed)**
3. **Phase 3: The Edge AI Engine (Completed)**

4. **Phase 4: Synthetic Data Seeding (Current)**
5. **Phase 5: The React Native Frontend**
6. **Phase 6: Integration & App Lifecycle**
7. **Phase 7: Native Biometric Ingestion**

User: Requested this document.